

4.1 素因数分解 (ショアのアルゴリズム)

4.1.1 位数発見問題として解く

位相推定サブルーチンの重要な応用例として、ショアの素因数分解アルゴリズムを紹介します。素因数分解問題とは整数 N が与えられたときに N の 1 でない約数を見つける問題です。これは簡単そうに見えますが、対象となる数が小さい場合だけです。例えば、 $15 = 3 \times 5$ は即答できますが、 $3007 = 31 \times 97$ 、 $283321 = 311 \times 911$ と数が大きくなるにつれ急速に難しくなります。 N がある整数 m で割り切れるかどうかは効率よくチェックできるので、単純には $2 \leq m < \sqrt{N}$ の整数 m で総当たりすれば解けるはずですが、この方法では、 N の桁が 1 つ増えるごとに組合せは 10 倍に増えるので、解を探すのに $\sqrt{10}$ 倍の時間がかかってしまいます (この方法の計算量は n 桁の整数に対して $\sqrt{10^n}$)。手計算と古典コンピュータによる計算は、計算量の意味では困難さは同じです。

これよりは速く素因数分解を解くアルゴリズムとして準指数的・超多項式関数的な計算時間のアルゴリズムが知られており、現在のベストアルゴリズムは一般数体ふるい法 (General Number Field Sieve, GNFS) です。これまでに古典コンピュータによって素因数分解された最大の数字は 768 ビット (10 進 232 桁) で、多項式時間^{*1}の古典アルゴリズムは見つかっていません。

インターネットで広く利用されている RSA 暗号の安全性は、非常に大きな数の素因数分解がとても難しいことに支えられています。もし大きな数の素因数分解を効率よく実行する方法があるとすると、インターネット上のさまざまな通信が危険にさらされる可能性が出てきてしまいます。

ショアは 1994 年に、量子コンピュータを用いると n 桁の整数の素因数分解を $O(n^3)$ の多項式時間で解くことが可能であると示しました。この量子アルゴリズムの発表は多くの研究者を惹きつけただけでなく、量子コンピュータの及ぼす経済や社会への影響の大きさから社会的な興味の対象にもなりました。

素因数分解問題は一般的には位数発見問題に帰着させて解きます。位数とは整

*1 多項式時間アルゴリズムとは、解くべき問題サイズ n に対して時間計算量が n の多項式で $O(n^k)$ (k は正の定数) と表現できるアルゴリズムを指します。

数 N を法とする共通の因数をもたない整数 x ($x < N$) に対し

$$x^r \bmod N = 1 \quad (4.1)$$

を満たす最小の整数 r です ($a \bmod b$ は整数 a を整数 b で割った余りを出力する剰余演算)。中でも冪剰余 (modular exponentiation) と呼ばれる

$$f(a)_{x,N} := x^a \bmod N$$

という剰余演算 (N を法とした x の a -冪剰余) は暗号理論の分野で広く利用されています。これは、与えられた a から $f(a)_{x,N}$ の値を計算するのは易しいのに、その逆の $f(a)_{x,N}$ の値から指数 a を求めることはとても難しいという性質 (一方向性) に由来します。与えられた N と x から a を求める問題は離散対数問題と呼ばれ、問題サイズに対して多項式時間の古典アルゴリズムは未発見です。位数を利用して素因数分解を行うアルゴリズムは以下のように表せます。

- i) 素因数分解したい整数 N と互いに素な整数 x を用意する^{*a}。
- ii) $a(=0, 1, \dots)$ を引数とした冪剰余 $f_{x,N}(a) := x^a \bmod N$ を計算する。
- iii) $f_{x,N}(a)$ の周期 r ($=$ 位数) を見つける (r が奇数なら i) に戻る^{*b})。
- iv) $p = \gcd(x^{r/2} + 1, N)$, $q = \gcd(x^{r/2} - 1, N)$ が N の素因数。

*a ランダムに選んできた整数 x' に対して最大公約数 $\gcd(x', N) = 1$ であれば x' と N は互いに素です。 $\gcd(x', N) \neq 1$ であれば、 N の約数の 1 つが見つかったことになるので、今度は $N' = N / \gcd(x', N)$ の素因数分解問題を考えればよいことになります。

*b r は複数の近似値で得られるのでその中から偶数の r を選ぶか、それでもダメな場合はステップ i) に戻り別の x で再度計算します。

アルゴリズム中に頻繁に登場する $\gcd(a, b)$ は整数 a と整数 b の最大公約数を計算する関数で、ユークリッドの互除法で効率よく計算可能です。量子コンピュータによって計算を高速化する部分は、ステップ iii) です。

$N = 21$, $x = 11$ を例にとってみます ($\gcd(11, 21) = 1$)。冪剰余 $f_{11,21}(a)$ は

$$f_{11,21}(0) = 11^0 \bmod 21 = 1 \bmod 21 = 1$$

$$f_{11,21}(1) = 11^1 \bmod 21 = 11 \bmod 21 = 11$$

$$f_{11,21}(2) = 11^2 \bmod 21 = 121 \bmod 21 = 16$$